TaskApp

Guía paso a paso para crear una aplicación móvil híbrida que ayuda a organizar tareas académicas, revisar su prioridad y conservarlas en el navegador.

Componentes Ionic | Formulario reactivo | TypeScript | localStorage

Objetivo de la práctica
Construir una lista sencilla de tareas con título, descripción, prioridad y estado de completado. El proyecto se ejecuta localmente con Ionic y puede adaptarse a una entrega en GitHub.

Del proyecto vacío a una tarea tipada

Primero se crea la aplicación y se define la forma de los datos. Así el resto de la pantalla puede trabajar con una estructura clara.

1 Crear y ejecutar el proyecto

Se partió de una plantilla en blanco de Ionic con Angular y componentes standalone. En Windows se usan los comandos `.cmd` para evitar el bloqueo de scripts de PowerShell.

```
ionic.cmd start TaskApp blank --type=angular
cd TaskApp
ionic.cmd serve
```

El servidor abre la aplicación local en el navegador para poder revisar los cambios mientras se desarrolla.

2 Definir el modelo de tarea

El modelo indica qué datos se guardan. `NuevaTarea` contiene lo que se escribe en el formulario; `Tarea` agrega el identificador y el estado de completado.

```
type Prioridad = 'Alta' | 'Media' | 'Baja';

interface NuevaTarea {
  titulo: string;
  descripcion: string;
  prioridad: Prioridad;
}

interface Tarea extends NuevaTarea {
  id: number;
  completada: boolean;
}
```

Archivo: `src/app/models/tarea.ts`

¿Por qué separar los tipos? El formulario todavía no conoce el `id` ni el estado inicial. El servicio asigna esos valores al crear la tarea.

Un servicio mantiene las tareas al día

La página presenta la información; el servicio concentra las operaciones y la persistencia local.

3 Crear TareaService

El servicio usa `@Injectable({ providedIn: 'root' })` para que Angular lo pueda inyectar. Sus métodos consultan, agregan, completan y eliminan tareas.

Agregar

Recibe una `NuevaTarea`, asigna un id y empieza con `completada: false`.

Completar

Cambia el estado de la tarea cuyo id coincide.

Eliminar

Filtra la lista para quitar la tarea elegida.

Consultar

Devuelve una copia de la lista para mostrarla.

Archivo: `src/app/services/tarea.ts`

4 Persistir con localStorage

Al iniciar la aplicación, el servicio lee el valor de la clave `taskapp-tareas`. Cuando cambia la lista, la convierte a JSON y la guarda otra vez.

Acción del usuario → **Servicio actualiza lista** → **JSON en localStorage**

La persistencia es local al navegador y al dispositivo. No sincroniza la información con otros equipos.

Idea clave: separar la lógica hace que los métodos de la página sean breves y que el guardado se realice siempre que se modifica una tarea.

Capturar, validar y mostrar tareas

La pantalla combina controles Ionic con un formulario reactivo de Angular para ofrecer una interacción simple.

5 Validar los datos

`ReactiveFormsModule` y `FormBuilder` crean el formulario. El título es obligatorio y debe tener al menos cinco caracteres útiles; la descripción es opcional y la prioridad inicia en “Media”.

```
titulo: ['', [
  Validators.required,
  Validators.minLength(5)
]]
```

6 Construir la interfaz

La plantilla usa encabezado, contenido, lista, elementos, etiquetas, insignias, checkbox, botones, pie y botón flotante de Ionic.

- `ion-badge` muestra la prioridad.
- `ion-checkbox` cambia el estado.
- `ion-icon` identifica la acción de eliminar.

7 Conectar acciones y estados

1. Al guardar, la página envía los datos válidos al servicio y actualiza la lista.
2. Al marcar una tarea, el servicio alterna `completada`; una clase CSS tacha el texto.
3. Al pulsar eliminar, se quita la tarea por su id y se vuelve a mostrar la lista.
4. El botón flotante abre el formulario para agregar nuevas actividades.

8 Estilos adaptables

El SCSS define colores, tarjetas, insignias de prioridad y espaciado para pantallas angostas y anchas. Los campos tienen contraste para distinguir etiquetas, texto y placeholders.

Archivos: `src/app/home/home.page.ts`, `home.page.html` y `home.page.scss`.

Comprobar el flujo y preparar la entrega

Antes de compartir el proyecto, se ejecuta en el navegador y se revisan sus funciones principales.

Lista de comprobación funcional

- El formulario informa si el título está vacío o tiene menos de cinco caracteres.
- Una tarea válida aparece con título, descripción y prioridad.
- El checkbox marca y desmarca la tarea; completada, su texto se muestra tachado.
- El botón de eliminar quita la tarea seleccionada.
- Después de recargar el navegador, las tareas siguen guardadas.
- El diseño se revisa en una ventana estrecha y otra más ancha.

Ejecutar

```
npm.cmd install
ionic.cmd serve
```

Para validar la compilación: `npm.cmd run build`.

Archivos de apoyo

- `README.md`: instalación y ejecución.
- `.gitignore`: excluye `node_modules` y artefactos.
- `GUIA_DESARROLLO.md`: pasos del proyecto.

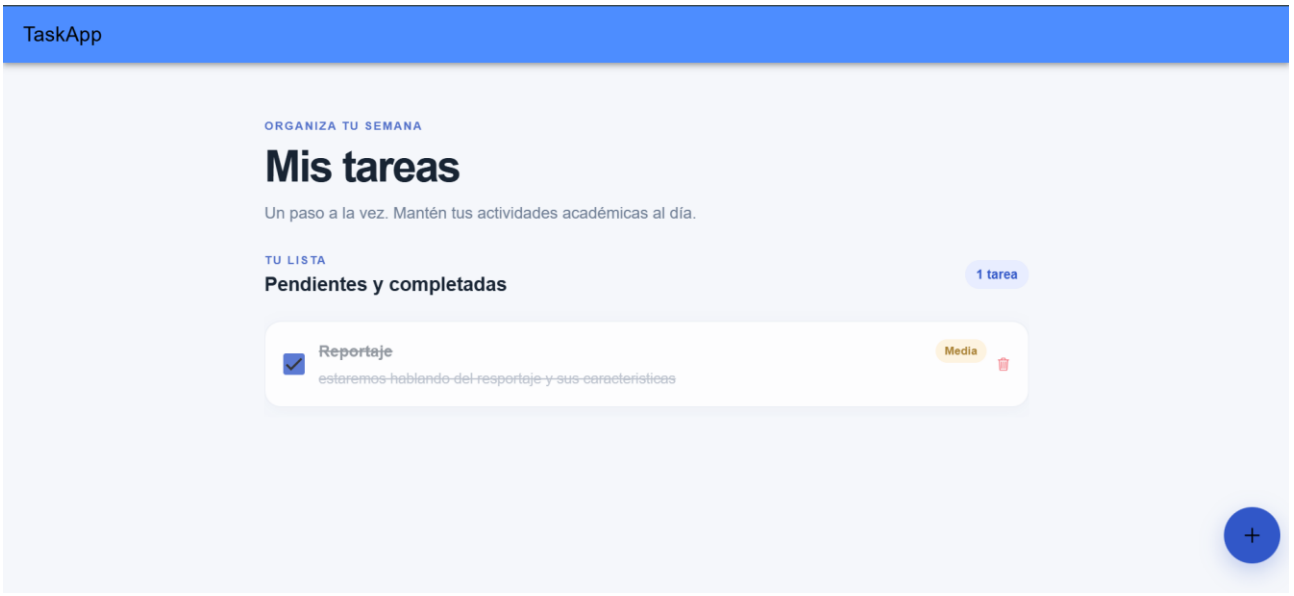
Preparar GitHub y las evidencias

Subir el código fuente junto con `package.json` y `package-lock.json`. No subir `node_modules`; Git lo omite mediante `.gitignore`.

Para complementar la entrega, añadir capturas de la pantalla vacía, los mensajes de validación, una tarea con su prioridad, una tarea completada y la terminal con la aplicación en ejecución.

Resultado: una aplicación académica pequeña, funcional y fácil de explicar. Cada parte tiene una responsabilidad: el modelo define datos, el servicio los gestiona y la página permite interactuar con ellos.

Aquí se puede apreciar la pagina principal de la pagina



Aca se puede verificar la creación de una tarea



En esta parte podemos ver como se crea y el botón de eliminar y también podemos tachar como completa

